

Café *Velours*

SOUTENANCE DE PROJET - TITRE CDPI

Grains d'Exception & Chocolats Premium | 2025-2026

| Sommaire de la Présentation

- > **01** Présentation & Concept
 - > **02** Expression des Besoins (MoSCoW)
 - > **03** Gestion de Projet : Backlogs
 - > **04** Prototypage UI/UX
 - > **05** Architecture : Écosystème Distribué
 - > **06** Build & Dépendances : Maven
 - > **07** Cycle de Vie Maven
 - > **08** Modélisation de Données : MCD
 - > **09** Modélisation du Logique au Physique
 - > **10** Front-End React 18
 - > **11** DevOps : Docker Compose Mesh
 - > **12** Infrastructure : MongoDB
 - > **13** Tests Unitaires Isolés
 - > **14** Couverture JaCoCo
 - > **15** Conclusion & Perspectives
-

| 01. Concept : Une Vitrine Sensorielle

Le mariage du luxe et du goût

Café Velours est un site E-commerce fictif de cafés d'exception et chocolat fins qui propose une immersion de goûts et arômes.

- Packs Duo exclusifs sélectionnés par experts.
- Storytelling axé sur la traçabilité.
- Expérience utilisateur fluide et premium.



| 02. Besoins : Méthode MoSCoW

La méthode MoSCoW (Must, Should, Could, Won't have) a permis de prioriser le backlog de Café Velours face à un temps de développement contraint.

Elle distingue ce qui est indispensable à la mise en production



Must Have

Panier persistant, Auth BCrypt, Tunnel de commande et règle de remise automatique

RG-07.



Should Have

Filtrage multi-critères, Recherche prédictive, Système d'avis clients avec notation.



Could Have

Mode sombre natif, Newsletter, Recommandations personnalisées via MongoDB.

**Won't have", sont les fonctionnalités explicitement exclues du périmètre
(ex : paiement multi-devises, application mobile native)**



| 03. Gestion de Projet : Backlog Agile

BACKLOG GITHUB — USER STORIES

Le backlog est organisé en sprints. Chaque user story suit le format : En tant que [rôle], je veux [action] afin de [bénéfice].

#	Sprint	User Story	Priorité	Criteres d'Acceptation	Statut
US-01	Sprint 1	En tant que visiteur, je veux voir le catalogue des cafés afin de découvrir les produits disponibles	MUST	Request HTTP GET asynchrone via Axios vers product-service (port 8082). Rendu dynamique sous forme de grille adaptative(Grid/Flexbox Tailwind CSS)	Fait
US-02	Sprint 1	En tant que visiteur, je veux voir la fiche détaillée d'un produit afin de connaître son origine et ses notes aromatiques	MUST	Passage d'un identifiant produit unique par les props de React. Rendu des	Fait

Pilotage par les Sprints

Utilisation de GitHub Projects pour un suivi granulaire de la valeur métier.

- User Stories normées (Rôle/Action/Bénéfice).
- Découpage en 4 Sprints itératifs.
- Traçabilité totale des critères d'acceptation.

04. Prototypage : UX/UI Design

Du Wireframe au Mockup

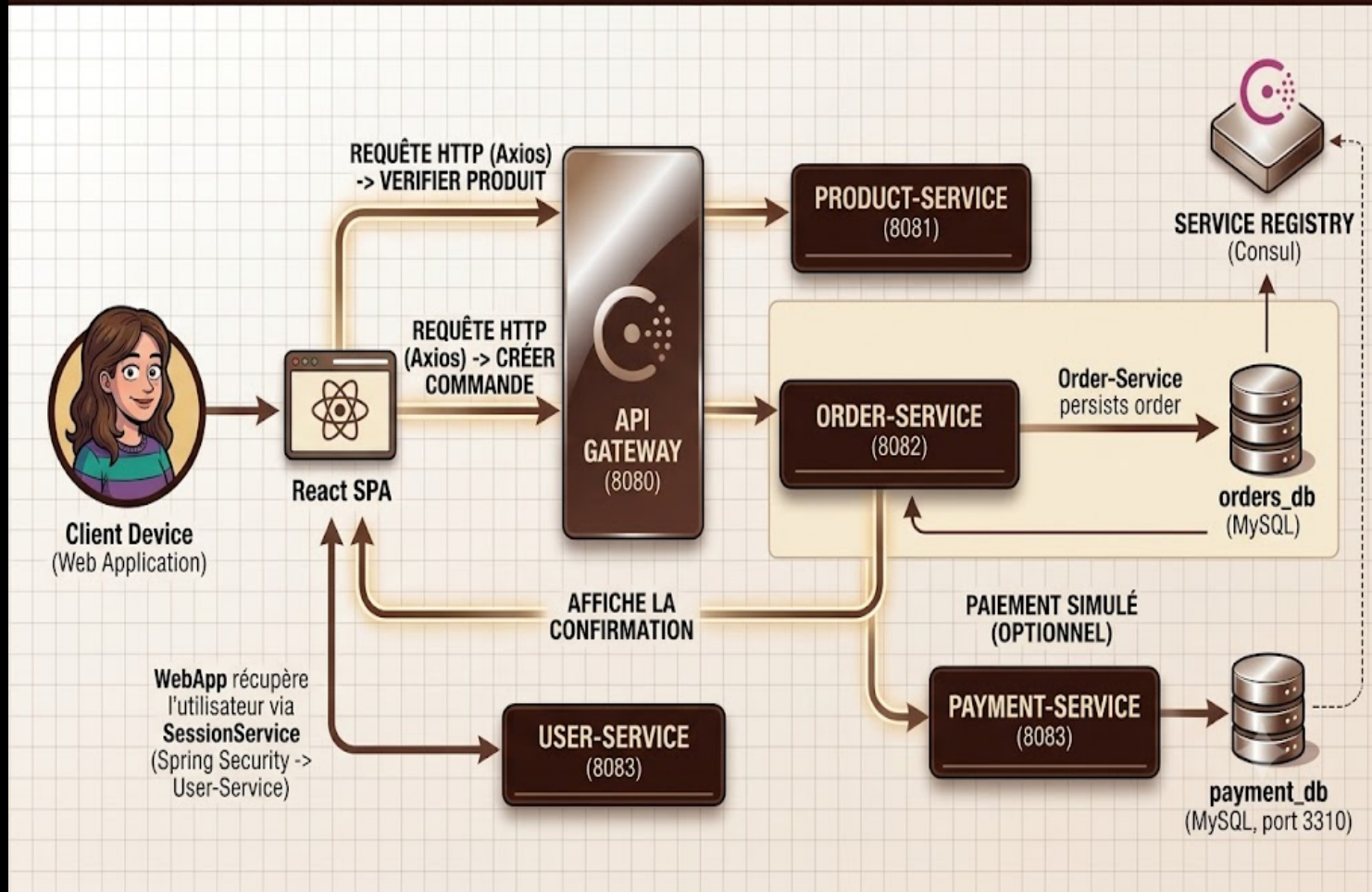
Processus de design centré utilisateur pour une ergonomie optimale.

- **Zonage** : Hiérarchie visuelle stricte.
- **Fidélité** : Design System "Luxe & Tech".
- **Contrastes** : Accents cuivrés (#B87333).



05. Architecture : Écosystème Distribué

FLUX DE CRÉATION D'UNE COMMANDE - CAFÉ VELOURS



Modularité & Résilience

- > **API Gateway** : Routage et sécurité.
- > **Consul** : Discovery Registry.
- > **Microservices** : Indépendance métier.
- > **Feign** : Communication synchrone.

06. Build & Dépendances : Apache Maven

- > **Maven** : Outil de build : compile, teste et package chaque microservice.
- > **pom.xml** : Descripteur XML : Group ID, Artifact ID, Version et dépendances.
- > Modules : 4 projets Maven indépendants : gateway, order, payment, product-service.

Build & Gestion des Dépendances

```
index.html pom.xml (order-service) x HomeView.jsx OrderServiceTest.java OrderServiceTest.class OrderControllerTest.class
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <parent>
6         <groupId>org.springframework.boot</groupId>
7         <artifactId>spring-boot-starter-parent</artifactId>
8         <version>3.5.14</version>
9         <relativePath/> </parent>
10    <groupId>com.cafevelours</groupId>
11    <artifactId>order-service</artifactId>
12    <version>0.0.1-SNAPSHOT</version>
13    <name>order-service</name>
14    <description>Order Service for Cafe Velours</description>
15
16    <properties>
17        <java.version>17</java.version>
18        <spring-cloud.version>2025.0.0</spring-cloud.version>
19    </properties>
20
21    <dependencies> Add Starters...
22    <dependency>
23        <groupId>org.springframework.boot</groupId>
24        <artifactId>spring-boot-starter-data-jpa</artifactId>
25    </dependency>
26    <dependency>
27        <groupId>org.springframework.boot</groupId>
28        <artifactId>spring-boot-starter-web</artifactId>
29    </dependency>
30    <dependency>
31        <groupId>org.springframework.cloud</groupId>
32        <artifactId>spring-cloud-starter-consul-discovery</artifactId>
33    </dependency>
```


| 07. Cycle de Vie Maven

Le cycle de vie Maven automatise et standardise la construction du projet à travers une séquence de phases obligatoires : de la validation du `pom.xml` jusqu'au déploiement du `.jar`, en passant par la compilation, l'exécution des tests et le contrôle qualité



Validate & Compile

validate : vérifie la structure du **pom.xml**.

compile : transforme le code source Java en bytecode (`.class`).



Test & Package

test : exécute les tests JUnit/Mockito.

package : génère le `.jar` exécutable de chaque microservice.



Verify & Deploy

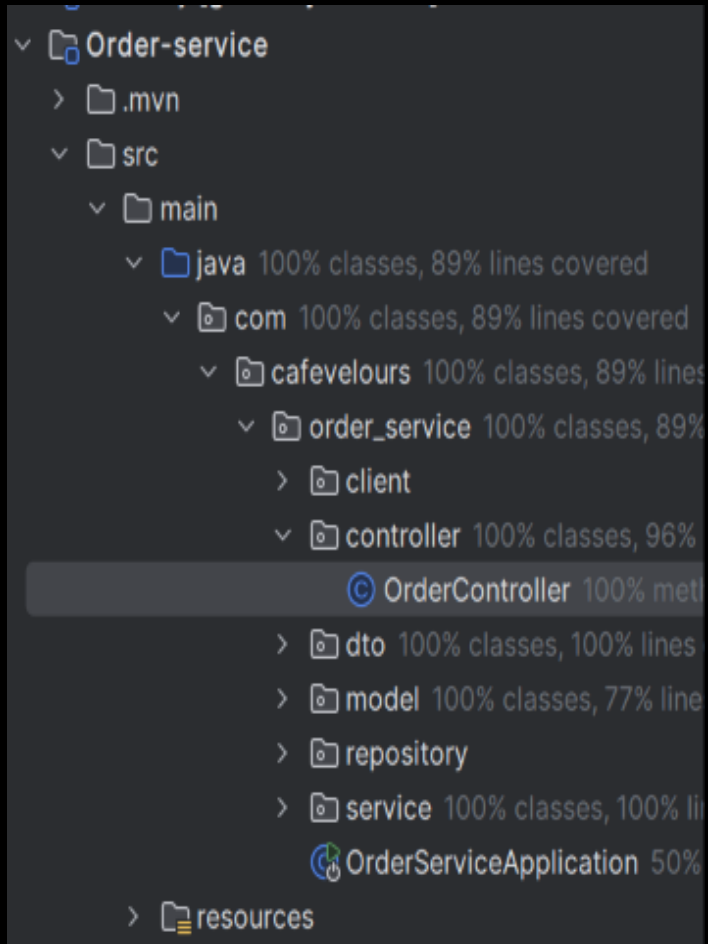
verify : contrôle la qualité (JaCoCo). install :

copie le `.jar` dans le repo local `~/.m2`.

deploy : publie sur un dépôt distant.

Chaque commande (`mvn install`) déclenche automatiquement toutes les phases précédentes — impossible d'empaqueter un `jar` sans que les tests soient passés.

Architecture de couches – Controller/Service/Repo



Chaque couche a un rôle unique et ne doit pas empiéter sur celui des autres.

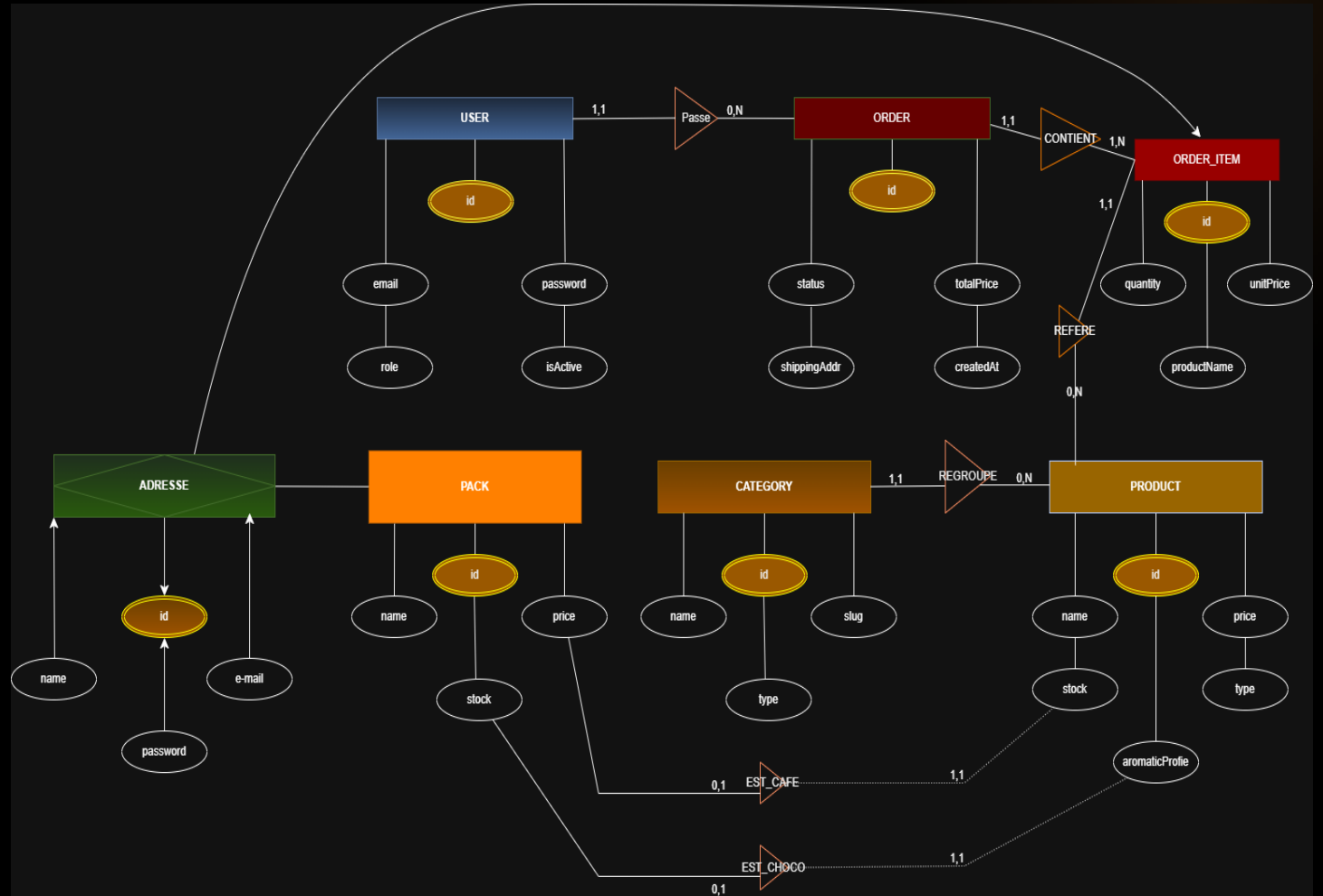
	Controller	Service	Repository
Rôle	Point logique entrée HTTP	Logique métier	Accès aux données
Fait	Reçoit/renvoie les requêtes REST	Calculs, règles de gestion	CRUD via Spring Data JPA
Exemple	@PostMapping("/orders")	createOrder()	findByUser_Id()
Ne fais pas	Pas de logique métier	Pas de code HTTP	Pas de SQL manuel

| o8. Modélisation de Données : MCD

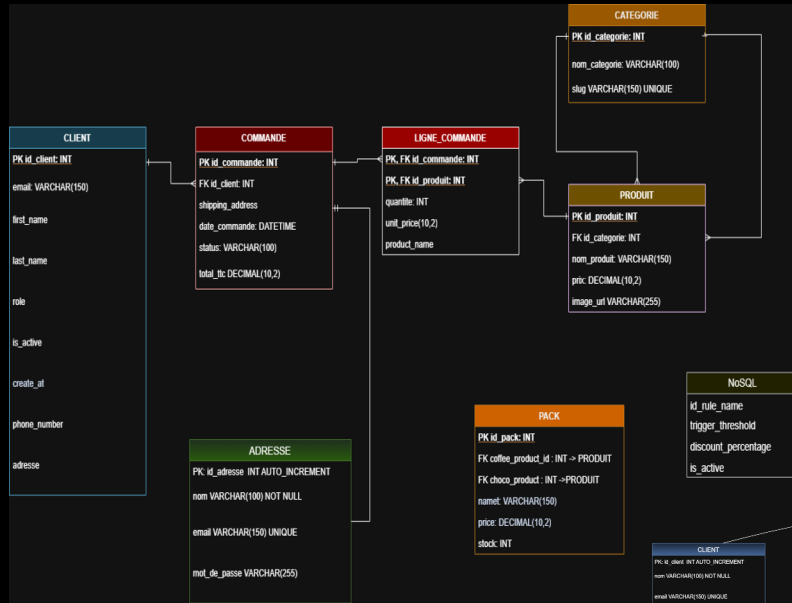
Règles Métier & Entités

Traduction sémantique de l'univers Café Velours.

- Entités clés : Client, Produit, Commande.
- Cardinalités strictes pour la cohérence.
- Structure optimisée pour les Packs Duo.

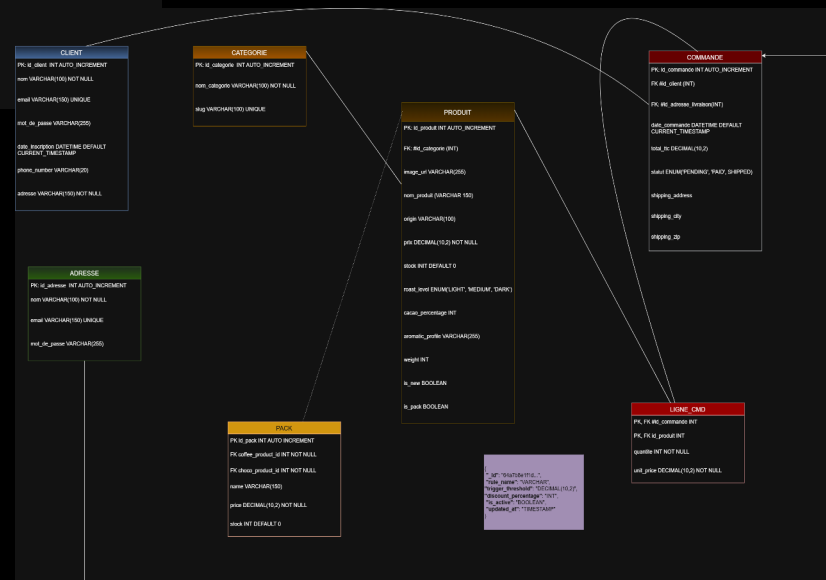


09. Modélisation : Du Logique au Physique



MLD

MPD



Normalisation Relationnelle

- Tables normalisées (3FN).
- Contraintes d'intégrité référentielle (FK).
- Scripts SQL MySQL 8 optimisés.
- Persistance hybride : MySQL + MongoDB.

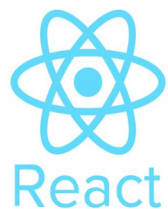
| 10. Front-End : React, rôle de Axios et Webapp

Performance & UX Fluide

Axios est la bibliothèque JavaScript utilisée pour effectuer les appels HTTP entre le frontend React et le backend (via l'API Gateway sur le port 8080). Concrètement :

- Abstraction de fetch() : Axios simplifie l'écriture des requêtes GET/POST/PUT/DELETE, gère automatiquement la sérialisation JSON (pas besoin de faire `JSON.stringify()/res.json()` manuellement)
- Gestion fine des événements : `e.stopPropagation()` pour isoler les actions.
- Toutes les requêtes Axios pointent vers le Gateway sur le port 8080, qui doit être démarré au préalable (avec les microservices enregistrés dans Consul)

```
1  import React, { useState } from 'react';
2
3  export default function HomeView({ onNavigate, onViewProduct }) { Show
4
5      // Données statiques des packs promotionnels mis en avant
6      const featuredPacks : [{id: number, name: string, pr... = [
7          { id: 23, name: 'Pack Guatemala', price: '26,50$', img: 'https:
8          { id: 20, name: 'Duo Intense Éthiopie', price: '27,00$', img: '
9          { id: 19, name: 'Duo Signature Colombie', price: '25,00$', img:
10         { id: 21, name: 'Duo Douceur Brésilienne', price: '24,50$', img
11         { id: 22, name: 'Duo Exploration Épicée', price: '28,00$', img:
12     ];
13     // Données statiques des packs promotionnels mis en avant
14     const featuredProducts : [{id: number, name: string, ty... = [
15         { id: 3, name: 'Finca el Paraíso', type: 'Café', price: '18,90$
16         { id: 16, name: 'Noir Fleur de Sel & Piment', type: 'Chocolat',
17         { id: 5, name: 'Virgacheffe Héritage', type: 'Café', price: '16
18         { id: 15, name: 'Lait Gourmand & Noisettes', type: 'Chocolat',
19     ];
20
```



| 11. DevOps : Docker Compose Mesh

Infrastructure Reproducible

Docker Compose orchestre en une seule commande (`docker compose up`) l'ensemble de l'infrastructure de Café Velours : les 2 instances MySQL isolées, la base MongoDB, et le registre Consul. Sans lui, chaque développeur devrait démarrer et configurer manuellement 4 conteneurs séparés avec les bons ports, réseaux et volumes. Il garantit ainsi une infrastructure reproductible — identique en local et en production — et élimine le classique "ça marche sur ma machine".

- > **Google Jib** : Builds optimisés sans Dockerfile.
- > **Isolation** : Réseau privé bridge virtuel.
- > **Healthchecks** : Ordonnancement intelligent du boot des conteneurs.

```
1  version: '3.8'
2
3  services:
4    # 1. L'ANNUAIRE DE SERVICES (HASHICORP CONSUL)
5    consul:
6      image: hashicorp/consul:1.15
7      container_name: consul-server
8      restart: always
9      ports:
10       - "8500:8500"          # Interface Web & API HTTP
11       - "8600:8600/udp"      # Résolution DNS interne
12      command: "agent -dev -client=0.0.0.0"
13      networks:
14       - cafe-velours-network
15
16    # 2. BASE DE DONNÉES PRINCIPALE (MYSQL - PRODUITS ET COMMANDES)
17    mysql-db:
18      image: mysql:8.0
19      container_name: cafe-velours-mysql
20      restart: always
21      ports:
22       - "3310:3306"          # Redirection externe 3310 pour éviter les conflits hôte
23      environment:
24        MYSQL_ROOT_PASSWORD: password
25        MYSQL_DATABASE: cafe_velours_db
26      volumes:
27       - mysql_data:/var/lib/mysql
28      networks:
29       - cafe-velours-network
30
```

| 12. Infrastructure : Conteneurs Docker

Orchestration Multi-Conteneurs

- > **docker-compose.yml** : Décrit et démarre tous les services d'un seul coup (up -d).
- > **Services** : Consul, MySQL et MongoDB tournent chacun dans leur propre conteneur.
- > Réseau & Volumes : Réseau bridge partagé et volumes persistants pour les données.

☐	▼	●	docker	-	-	-	7.03%	979.93MB / 30.64	12.49%	501	■	⋮	🗑
☐		●	consul-server	4b9e85cbaf44	hashicorp/consul	8500:8500 ↗ Show all ports (2)	2.5%	30.13MB / 7.66Gi	0.38%	120	■	⋮	🗑
☐		●	cafe-velours-mongo	0751e1288559	mongo:latest	27017:27017 ↗	2%	206.1MB / 7.66Gi	2.63%	206	■	⋮	🗑
☐		●	cafe-velours-mysql	62344df017f0	mysql:8.0	3310:3306 ↗	0.99%	379MB / 7.66GB	4.83%	99.2	■	⋮	🗑
☐		●	payment-db	312de9172abf	mysql:8.0	3312:3306 ↗	1.54%	364.7MB / 7.66Gi	4.65%	76M	■	⋮	🗑

Docker Desktop — conteneurs actifs du réseau Café Velours : Consul, MySQL (x2) et MongoDB.

| 12. Role de MongoDB

Persistence Polyglotte

- > Les règles de remises(-10% de remise dès 50euros d'achat) sont de règles métier flexibles et evolutives
- > Le schéma dynamique e MongoDB (document JSON) permet de stocker ces règles sans contrainte des colonnes figées, contrairement à une table SQL rigide
- > Réseau & Volumes : Réseau bridge partagé et volumes persistants pour les données.

trigger_treshold :le montant planche du panier à partir duquel la remise s'active

discount_percentage : le taux de réduction appliqué

is_active : permet d'activer/désactiver une règle sans la supprimer (utile pour des opérations commerciales temporaires)

```
json
{
  "_id": "64a7b8e1f1d...",
  "rule_name": "Remise Panier 50€",
  "trigger_threshold": 50.00,
  "discount_percentage": 10,
  "is_active": true,
  "updated_at": "2026-07-08T10:00:00Z"
}
```

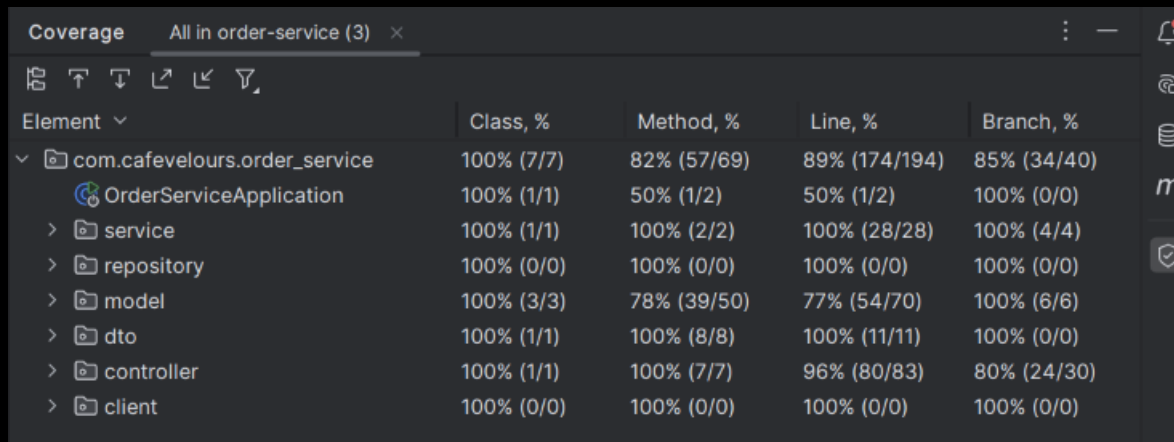

| 13. Qualification : Tests Unitaires Isolés

Validation des Règles Métier

- **JUnit 5** : Moteur de test standard.
- **Mockito** : Simulation des bases de données MySQL et MongoDB.
- Validation mathématique de la remise automatique **RG-07**.

La stratégie de test de Café Velours repose sur l'isolation totale de la couche métier :

chaque composant est validé indépendamment de ses dépendances externes (bases de données, appels réseau), garantissant que les règles de gestion restent fiables même en cas de panne d'un service tiers.



The screenshot shows a code coverage report for the package 'com.cafevelours.order_service'. The report is displayed in a table with columns for 'Element', 'Class, %', 'Method, %', 'Line, %', and 'Branch, %'. The data shows that the 'com.cafevelours.order_service' package has 100% class coverage (7/7), 82% method coverage (57/69), 89% line coverage (174/194), and 85% branch coverage (34/40). The sub-packages 'OrderServiceApplication', 'service', 'repository', 'model', 'dto', 'controller', and 'client' all have 100% class coverage. The 'controller' package has 100% method coverage (7/7), 96% line coverage (80/83), and 80% branch coverage (24/30). The 'client' package has 100% method coverage (0/0), 100% line coverage (0/0), and 100% branch coverage (0/0).

Element	Class, %	Method, %	Line, %	Branch, %
com.cafevelours.order_service	100% (7/7)	82% (57/69)	89% (174/194)	85% (34/40)
OrderServiceApplication	100% (1/1)	50% (1/2)	50% (1/2)	100% (0/0)
service	100% (1/1)	100% (2/2)	100% (28/28)	100% (4/4)
repository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
model	100% (3/3)	78% (39/50)	77% (54/70)	100% (6/6)
dto	100% (1/1)	100% (8/8)	100% (11/11)	100% (0/0)
controller	100% (1/1)	100% (7/7)	96% (80/83)	80% (24/30)
client	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)

| 14. Qualimétrie : Couverture JaCoCo

89 % de couverture de lignes sur order-service — 33 tests unitaires, 0 échec.

Product-Service (@Service)

85 %

Order-Service (@Service)

89 %

User-Service (@Service)

78 %

Seuil de tolérance atteint : Couverture globale > 80% sur la logique métier critique.

| 15. Conclusion & Perspectives

"Le MoSCoW, c'était mon arbitrage fonctionnel de départ — ce que je choisisais de ne pas développer par manque de temps. Les perspectives, ce sont les évolutions techniques identifiées maintenant que le MVP est fonctionnel : passer d'une simulation de paiement à une vraie intégration Stripe, ou industrialiser le déploiement."

ETAPE	MOSCOW	CONCLUSION - PERSPECTIVES
Question	Quelles fonctionnalités utilisateur manquent ?	Quelles évolutions techniques/infra sont prévues ?
Nature	Besoins métier priorisés en amont du projet	Axes de maturation post-MVP
Exemple	Mode sombre, newsletter, recommandations IA	Paiement réel (Stripe), déploiement Kubernetes
Decidé	Avant le développement (phase de cadrage)	Après le développement (retour d'expérience)

FIN DE LA PRÉSENTATION

Questions & *Réponses*

Merci pour votre attention.

☉ CAFÉ VELOURS | TITRE CDPI 2026
